Procurement and Inventory Analytics Agent - POC Framework

Client: Manufacturing Plant (21C)
Objective: Build an AI-powered Natural Language Query interface for Procurement and Inventory Analytics
Current Data Source: Static Excel files (POC) → Infor ERP (Production)

---

1. EXECUTIVE SUMMARY

1.1 What You're Building

An intelligent agent that allows business users (procurement managers, inventory planners, operations teams) to query inventory and procurement data using plain English questions instead of complex SQL or manual Excel analysis.

1.2 Business Value

- **Faster Decision Making**: Instant answers vs hours of manual analysis
- **Democratized Analytics**: Non-technical users can access insights
- **Proactive Alerts**: Agent can flag critical issues automatically
- **Cost Reduction**: Optimize inventory levels, reduce stockouts and excess inventory
- **Supplier Performance**: Identify delivery delays and quality issues early

---

2. CURRENT DATA ASSESSMENT

2.1 Available Data (Your Excel Files)

File 1: 21c_3yr_MRP_items_consumption.xlsx

- **Records**: 426 materials
- **Key Fields**:
  - Sr No (Sequential ID)
  - Item (Material Code) - e.g., 10401
  - Item Description - e.g., "ADHESIVE,RUBBER,PC-65 DENDRITE"
  - Consumption FY 2022-23
  - Consumption FY 2023-24
  - Consumption FY 2024-25
  - Total 3 YR consumption
  - Safety Stock (21C Plant)

File 2: Lead_Time__21c_3yr_MRP_items_consumption.xlsx

- **Records:** 426 materials (same as File 1)
- **Additional Field:**
  - Lead Time (in Days) - e.g., 7, 15, 30 days

2.2 Data Coverage Matrix

Data Category	Available in Current Data	Missing from Current Data
Material Master	✅ Material Code ✅ Description ✅ Safety Stock	❌ Unit of Measure ❌ Material Category/Group ❌ ABC Classification ❌ Unit Price/Cost ❌ Criticality Flag
Consumption	✅ Yearly consumption (3 years) ✅ Total consumption	❌ Monthly breakdown ❌ Seasonal patterns ❌ Production order linkage ❌ Actual vs Planned
Inventory	⚠️ Safety Stock only	❌ Current stock on hand ❌ Available stock ❌ Stock in transit ❌ Reserved stock ❌ Stock location ❌ Stock value ❌ Movement history
Procurement	⚠️ Lead time only	❌ Purchase orders ❌ PO quantity, dates ❌ GRN (Goods Receipt) ❌ Payment terms ❌ Price history
Supplier	❌ None	❌ Supplier master ❌ Supplier ratings ❌ Delivery performance ❌ Quality metrics ❌ Contact details
Production	❌ None	❌ Production orders ❌ BOM (Bill of Materials) ❌ Work orders ❌ Machine downtime

Legend: ✅ Available | ⚠️ Partial | ❌ Missing

---

3. POC SCOPE - WHAT YOU CAN BUILD NOW

3.1 Achievable with Current Data

Consumption Analytics ⭐⭐⭐

- Monthly/Yearly consumption trends per material
- Year-over-Year growth rates
- Consumption forecasting (basic time series)
- Top/Bottom consumers
- Consumption volatility analysis

Inventory Planning ⭐⭐

- Safety stock adequacy assessment
- Reorder point calculation (using lead time + consumption)
- Days of inventory on hand (theoretical)
- Fast/Slow moving classification based on consumption

Material Insights ⭐⭐

- Material categorization by consumption patterns
- Criticality scoring based on consumption + lead time
- Search and filter by description keywords

3.2 Sample Queries You Can Answer NOW

```
✅ "Show me top 20 materials by consumption in 2024-25"
✅ "Which materials have consumption growth > 20% year-over-year?"
✅ "List materials with lead time > 20 days and high consumption"
✅ "Calculate reorder points for all materials"
✅ "Which materials are slow-moving in the last 2 years?"
✅ "Show me materials with 'BEARING' in description"
✅ "What's the average consumption trend for CHAIN items?"
✅ "Forecast consumption for next year for Item 10488"
✅ "Which materials have volatile consumption patterns?"
✅ "Show materials with safety stock < 10 days of consumption"
```

3.3 Queries You CANNOT Answer Yet (Need More Data)

```
❌ "Show me slow-moving materials over the last six months"
   → Need: Monthly inventory movement data from Infor

❌ "Why has inventory value increased this quarter?"
   → Need: Current stock levels + Material cost data

❌ "Which suppliers frequently delay deliveries?"
   → Need: PO data, GRN data, Supplier master

❌ "Which materials are currently critical to production?"
   → Need: Current stock, Production orders, BOM data
```

---

4. CRITICAL DATA GAPS TO FILL

Priority 1: MUST HAVE for Production Agent 🔴

4.1 Current Inventory Snapshot (Daily/Real-time from Infor)

```
Required Fields:
- Material_Code
- Plant_Code
- Storage_Location
- Unrestricted_Stock (Available)
- Quality_Inspection_Stock
- Blocked_Stock
- Stock_in_Transit
- Reserved_Stock
- Stock_Value (Currency)
- Last_Movement_Date
- Stock_Unit (EA, KG, MTR, etc.)
```

Why Critical: Without current stock, you can't identify actual shortages, excess, or slow-moving items.

Infor Table/API:

- `ICITEM` (Item Master)
- `ICLOCATION` (Item Location)
- `ICITEMBAL` (Item Balance)

4.2 Purchase Order Data (Transactional - Daily updates)

```
Required Fields:
- PO_Number
- PO_Line_Number
- Material_Code
- Supplier_Code
- Supplier_Name
- Order_Quantity
- Received_Quantity
- Outstanding_Quantity
- Unit_Price
- Currency
- Order_Date
- Promised_Delivery_Date
- Actual_Delivery_Date
- PO_Status (Open/Closed/Cancelled)
- GRN_Number (Goods Receipt Note)
- GRN_Date
```

Why Critical: Essential for supplier performance analysis and delivery tracking.

Infor Table/API:

- `POLINE` (Purchase Order Line)
- `PORELEASEDETAIL` (PO Release)
- `ICDISTRIBUTION` (Receipts/Issues)

4.3 Supplier Master Data

```
Required Fields:
- Supplier_Code
- Supplier_Name
- Contact_Person
- Email
- Phone
- Address
- Payment_Terms
- Currency
- Active_Status
- Supplier_Type (Local/Import)
- Lead_Time_Default
```

Why Critical: Link POs to suppliers for performance analysis.

Infor Table/API:

- `APVENDOR` (Vendor Master)
- `APVENMAST` (Vendor Master Details)

4.4 Material Master Data Enrichment

```
Required Fields (Beyond what you have):
- Material_Code
- Unit_of_Measure (EA, KG, L, etc.)
- Material_Group / Category
- ABC_Classification
- XYZ_Classification
- Standard_Price / Moving_Average_Price
- Criticality_Flag (A/B/C)
- Min_Order_Quantity
- Max_Stock_Level
- Reorder_Level
- MRP_Type (Manual/Automatic)
- Procurement_Type (Stock/External/Production)
```

Why Critical: Enables ABC/XYZ analysis, value-based prioritization.

Infor Table/API:

- `ICITEM` (Item Master)
- `ICSPEC` (Item Specifications)

Priority 2: SHOULD HAVE for Enhanced Analytics 🟡

4.5 Inventory Movement History (Time-series data)

```
- Material_Code
- Movement_Type (Receipt/Issue/Transfer)
- Quantity
- Movement_Date
- Document_Number (PO/Production Order/Transfer Order)
- From_Location / To_Location
- Batch_Number (if applicable)
```

Why: Enables monthly consumption trends, seasonal analysis, aging analysis.

Infor Table/API:

- `ICDISTRIBUTION` (Distribution History)
- `ICTRANS` (Transaction History)

4.6 Production Order Data

```
- Production_Order_Number
- Material_Code (Finished Goods)
- Planned_Quantity
- Actual_Quantity
- Order_Start_Date
- Order_End_Date
- Status
- Material_Components (BOM linkage)
```

Why: Links consumption to production, identifies critical materials.

Infor Table/API:

- `SFPRODUCTION` (Shop Floor Production)
- `SFORDER` (Production Orders)

4.7 Bill of Materials (BOM)

```
- Parent_Material_Code
- Component_Material_Code
- Quantity_Per_Assembly
- BOM_Level
- Scrap_Percentage
```

Why: Enables "Where Used" analysis, impact assessment.

Infor Table/API:

- `ENGBOM` (Engineering BOM)
- `ENGCOMPONENT` (BOM Components)

Priority 3: NICE TO HAVE for Advanced Features 🟢

- **Quality Inspection Data**: Rejection rates, quality scores
- **Supplier Rating/Scorecards**: Performance metrics

- **Contract Management**: Price agreements, volume commitments
- **Demand Planning**: Sales forecasts, production plans
- **Warehouse Data**: Bin locations, picking sequences

---

5. POC ARCHITECTURE

5.1 Technology Stack Recommendation

```
┌─────────────────────────────────────────────────────────────┐
│                    USER INTERFACE                           │
│  - Streamlit / Gradio (Quick POC)                           │
│  - Claude.ai Artifact (Interactive Demo)                    │
│  - Slack Bot (Future)                                       │
└─────────────────────────────────────────────────────────────┘
                          │
                          ▼
┌─────────────────────────────────────────────────────────────┐
│              AI AGENT ORCHESTRATION                         │
│  - LangChain / LlamaIndex (Agentic framework)               │
│  - Claude Sonnet 4 (NLU & Code Generation)                  │
│  - Prompt Templates (Query Understanding)                   │
└─────────────────────────────────────────────────────────────┘
                          │
                          ▼
┌─────────────────────────────────────────────────────────────┐
│              DATA LAYER (POC)                               │
│  - SQLite Database (Loaded from Excel)                      │
│  - Pandas DataFrames (In-memory)                            │
│                                                             │
│              DATA LAYER (PRODUCTION)                        │
│  - Infor CloudSuite API / ODBC Connection                   │
│  - PostgreSQL / Data Warehouse (Cached layer)               │
└─────────────────────────────────────────────────────────────┘
                          │
                          ▼
┌─────────────────────────────────────────────────────────────┐
│              ANALYTICS ENGINE                               │
│  - Python: Pandas, NumPy, SciPy                             │
│  - Visualization: Plotly, Matplotlib                        │
│  - Forecasting: Prophet, statsmodels                        │
│  - ML Models: Scikit-learn (Classification, Clustering)     │
└─────────────────────────────────────────────────────────────┘
```

5.2 Component Breakdown

Frontend (User Interface)

- **Option 1 - Streamlit** (Recommended for POC)
  - Pros: Fast development, Python-native, easy deployment
  - Cons: Limited customization
- **Option 2 - Gradio**
  - Pros: Even simpler than Streamlit, great for AI demos
  - Cons: Less control over layout
- **Option 3 - Claude Artifact**
  - Pros: No deployment needed, instant demo
  - Cons: Not production-ready

AI Agent Layer

- **LangChain Components**:
  - **Agent**: ReAct agent or SQL Agent
  - **Tools**:
    - `query_database`: Execute SQL queries
    - `analyze_trend`: Time-series analysis
    - `forecast_consumption`: Predictive analytics
    - `calculate_metrics`: KPI calculations
    - `visualize_data`: Chart generation
  - **Memory**: Conversation history, user preferences
  - **Retrieval**: Semantic search on material descriptions

Data Processing

- **ETL Pipeline** (For POC):
  - Excel → Pandas → SQLite
  - Data validation and cleaning
  - Feature engineering (calculated columns)
- **ETL Pipeline** (For Production):
  - Infor API → Incremental load → PostgreSQL
  - Daily batch + Real-time streaming
  - Data quality checks

Analytics Capabilities

1. **Descriptive Analytics**: What happened?

2. **Diagnostic Analytics**: Why did it happen?
3. **Predictive Analytics**: What will happen?
4. **Prescriptive Analytics**: What should we do?

---

6. POC IMPLEMENTATION ROADMAP

Phase 1: Data Preparation (Week 1)

Deliverable: Clean, structured database

Tasks:

1. Load Excel files into SQLite database
2. Create unified schema:

```sql
CREATE TABLE material_master (
    material_code TEXT PRIMARY KEY,
    description TEXT,
    safety_stock REAL,
    lead_time_days INTEGER,
    created_date DATE
);

CREATE TABLE consumption_history (
    id INTEGER PRIMARY KEY,
    material_code TEXT,
    fiscal_year TEXT,
    consumption_qty REAL,
    FOREIGN KEY (material_code) REFERENCES material_master(material_code)
);
```

3. Add calculated fields:
   - Average monthly consumption
   - Consumption volatility (Coefficient of Variation)
   - YoY growth rate
   - Days of stock (Safety Stock / Avg Daily Consumption)
   - Reorder Point (Lead Time * Avg Daily Consumption + Safety Stock)
4. Create materialized views for common aggregations

Success Criteria:

- ✅ All 426 materials loaded
- ✅ Zero data quality issues
- ✅ Sub-second query performance

Phase 2: Basic NL Query Engine (Week 2)

Deliverable: Working agent that answers 5 predefined question types

Tasks:

1. Set up LangChain SQL Agent
2. Create prompt templates:

```python
template = """
You are a procurement analyst. User asked: {user_query}

Database schema:
{schema}

Generate SQL query to answer. Only return valid SQLite syntax.
"""
```

3. Implement 5 query patterns:
    - Top N materials by metric
    - Trend analysis (time-series)
    - Filtering by criteria
    - Comparative analysis
    - Aggregations and grouping
4. Add validation layer (prevent SQL injection, check results)
5. Build simple Streamlit UI:
    - Text input for questions
    - Display results as table + chart
    - Show generated SQL (for transparency)

Test Queries:

```
1. "Show top 10 materials by consumption in 2024-25"
2. "Which materials have >30% YoY growth?"
3. "List materials with lead time >20 days"
4. "Calculate average consumption for BEARING materials"
5. "Show consumption trend for item 10488 over 3 years"
```

Success Criteria:

- ✅ 90%+ accuracy on test queries
- ✅ Response time < 3 seconds
- ✅ Natural language understanding works

Phase 3: Advanced Analytics (Week 3)

Deliverable: Forecasting, classification, and proactive alerts

Tasks:

1. Implement time-series forecasting:
    - Simple Moving Average
    - Exponential Smoothing
    - Prophet model (seasonal patterns)
2. Add classification models:
    - ABC Analysis (by consumption value)
    - XYZ Analysis (by consumption variability)
    - Fast/Slow/Non-moving classification
3. Create alert system:
    - Materials below safety stock
    - High lead time + low safety stock (critical risk)
    - Unusual consumption spikes
4. Build dashboard view:
    - KPI cards (Total materials, Avg consumption, etc.)
    - Charts (Pareto, Trends, Heatmaps)
    - Drill-down capability

Test Queries:

```
1. "Forecast consumption for next 6 months for all CHAIN items"
2. "Classify all materials into ABC categories"
3. "Alert me about materials at risk of stockout"
4. "Show me materials with erratic consumption patterns"
```

Success Criteria:

- ✅ Forecast MAPE < 20%
- ✅ ABC classification matches business intuition
- ✅ Alerts are actionable

Phase 4: Integration Preparation (Week 4)

Deliverable: Production-ready architecture blueprint + Data extraction specs

Tasks:

1. Document Infor API/ODBC connection requirements
2. Create data extraction queries for all Priority 1 tables
3. Design incremental load strategy
4. Build mock data generator (simulates Infor feeds)
5. Test agent with expanded schema
6. Performance testing (handle 10K+ materials, 1M+ transactions)
7. Security & access control design
8. Deployment plan (Docker, Cloud, On-premise)

Success Criteria:

- ✅ Detailed Infor integration specs
- ✅ Agent works with 10x data volume
- ✅ Production deployment plan approved

---

7. SAMPLE AGENT INTERACTION FLOW

Example 1: Simple Query

```
User: "Show me top 10 materials by consumption in 2024-25"

Agent Process:
1. Parse intent: "Top N ranking query"
```

```
2. Identify entity: "materials"
3. Identify metric: "consumption"
4. Identify filter: "2024-25"
5. Identify limit: "10"

Generated SQL:
SELECT material_code, description, consumption_2024_25
FROM material_master
ORDER BY consumption_2024_25 DESC
LIMIT 10;

Output:
┌─────────────────┬──────────────────────────────────┬────────────────┐
│ Material Code   │ Description                      │ Consumption    │
├─────────────────┼──────────────────────────────────┼────────────────┤
│ 10488           │ CHAIN PITCH 1/2" No-08B-1        │ 2779.00        │
│ 10582           │ DIN RAIL MCB 36mm                │ 1780.80        │
│ ...             │ ...                              │ ...            │
└─────────────────┴──────────────────────────────────┴────────────────┘


+ Bar chart visualization
```

Example 2: Complex Multi-step Query

```
User: "Which materials have high consumption but low safety stock coverage?"

Agent Process:
1. Define "high consumption": Top 25% by avg consumption
2. Define "low safety stock coverage": < 30 days of stock
3. Calculate: Days of stock = Safety Stock / (Avg Annual Consumption / 365)

Generated Analysis:
Step 1: Calculate average daily consumption
Step 2: Calculate days of safety stock
Step 3: Filter materials with days < 30 and consumption > 75th percentile

Output:
⚠️ ALERT: 23 materials identified at risk

┌────────┬──────────────────────┬────────────┬──────────────┬────────────┐
│ Code   │ Description          │ Avg Daily  │ Safety Stk   │ Days       │
├────────┼──────────────────────┼────────────┼──────────────┼────────────┤
│ 10488  │ CHAIN PITCH 1/2"     │ 17.71      │ 164.15       │ 9.3 days   │
│ ...    │ ...                  │ ...        │ ...          │ ...        │
└────────┴──────────────────────┴────────────┴──────────────┴────────────┘
```

```
Recommendation: Consider increasing safety stock for these items.
```

---

8. DATA EXTRACTION GUIDE FROM INFOR

8.1 Access Methods

Option 1: ODBC Connection (Recommended for POC)

```python
import pyodbc

connection_string = """
Driver={Infor ODBC Driver};
Server=your_infor_server;
Database=your_database;
UID=your_username;
PWD=your_password;
"""

conn = pyodbc.connect(connection_string)
query = "SELECT * FROM ICITEM WHERE COMPANY = '21C'"
df = pd.read_sql(query, conn)
```

Option 2: Infor ION API (REST)

- More flexible, cloud-friendly
- Requires API credentials and OAuth setup
- Better for real-time data

Option 3: IPA (Infor Process Automation)

- Schedule batch exports
- CSV/Excel dumps to shared folder
- Simpler but less real-time

8.2 Sample Extraction Queries

Current Inventory

```sql
```

```sql
SELECT
    i.ITEM as material_code,
    i.DESCRIPTION as description,
    l.LOCATION as storage_location,
    l.ON_HAND as unrestricted_stock,
    l.ON_ORDER as on_order_qty,
    l.COMMITTED as reserved_qty,
    i.UNIT_COST as unit_cost,
    i.UM as unit_of_measure
FROM ICITEM i
LEFT JOIN ICLOCATION l ON i.ITEM = l.ITEM
WHERE i.COMPANY = '21C'
  AND l.LOCATION = 'MAIN'
```

Purchase Orders

```sql
SELECT
    p.PO_CODE as po_number,
    p.PO_RELEASE as po_line,
    p.ITEM as material_code,
    p.VENDOR as supplier_code,
    v.NAME as supplier_name,
    p.QTY_ORDERED as order_qty,
    p.QTY_RECEIVED as received_qty,
    p.UNIT_PRICE as unit_price,
    p.PO_DATE as order_date,
    p.DUE_DATE as promised_delivery_date,
    p.PO_STATUS as status
FROM POLINE p
LEFT JOIN APVENDOR v ON p.VENDOR = v.VENDOR
WHERE p.COMPANY = '21C'
  AND p.PO_STATUS IN ('O', 'P')  -- Open or Partially Received
```

Movement History (Last 6 months)

```sql
```

```sql
SELECT
    t.ITEM as material_code,
    t.TRANS_TYPE as movement_type,
    t.QUANTITY as quantity,
    t.TRANS_DATE as movement_date,
    t.DOCUMENT_NUM as document_number,
    t.FROM_LOC as from_location,
    t.TO_LOC as to_location
FROM ICTRANS t
WHERE t.COMPANY = '21C'
  AND t.TRANS_DATE >= DATEADD(month, -6, GETDATE())
ORDER BY t.TRANS_DATE DESC
```

8.3 Infor-Specific Considerations

- **Company Code:** Always filter by '21C' (your plant)
- **Date Formats:** Infor uses Julian dates in some tables
- **Unit Conversions:** Handle different UoMs (EA, KG, L, etc.)
- **Null Values:** Infor often uses spaces instead of NULL
- **Performance:** Use indexes on ITEM, COMPANY, TRANS_DATE

---

9. ENHANCED SCHEMA FOR PRODUCTION

```sql
```

```sql
-- Material Master (Extended)
CREATE TABLE material_master (
    material_code TEXT PRIMARY KEY,
    description TEXT,
    material_group TEXT,
    uom TEXT,
    abc_class TEXT,
    xyz_class TEXT,
    standard_price REAL,
    safety_stock REAL,
    min_order_qty REAL,
    max_stock_level REAL,
    reorder_level REAL,
    lead_time_days INTEGER,
    criticality TEXT,
    is_active BOOLEAN,
    created_date DATE,
    updated_date DATE
);

-- Inventory Current
CREATE TABLE inventory_current (
    id INTEGER PRIMARY KEY,
    material_code TEXT,
    plant_code TEXT,
    storage_location TEXT,
    unrestricted_stock REAL,
    quality_stock REAL,
    blocked_stock REAL,
    stock_in_transit REAL,
    reserved_stock REAL,
    available_stock REAL,
    stock_value REAL,
    last_movement_date DATE,
    snapshot_date DATE,
    FOREIGN KEY (material_code) REFERENCES material_master(material_code)
);

-- Purchase Orders
CREATE TABLE purchase_orders (
    id INTEGER PRIMARY KEY,
    po_number TEXT,
    po_line INTEGER,
    material_code TEXT,
    supplier_code TEXT,
    supplier_name TEXT,
```

```sql
    order_quantity REAL,
    received_quantity REAL,
    outstanding_quantity REAL,
    unit_price REAL,
    currency TEXT,
    order_date DATE,
    promised_delivery_date DATE,
    actual_delivery_date DATE,
    po_status TEXT,
    grn_number TEXT,
    grn_date DATE,
    delivery_variance_days INTEGER,
    FOREIGN KEY (material_code) REFERENCES material_master(material_code),
    FOREIGN KEY (supplier_code) REFERENCES supplier_master(supplier_code)
);

-- Supplier Master
CREATE TABLE supplier_master (
    supplier_code TEXT PRIMARY KEY,
    supplier_name TEXT,
    contact_person TEXT,
    email TEXT,
    phone TEXT,
    address TEXT,
    payment_terms TEXT,
    currency TEXT,
    is_active BOOLEAN,
    supplier_type TEXT,
    default_lead_time INTEGER,
    rating REAL
);

-- Consumption History (Monthly granularity)
CREATE TABLE consumption_history (
    id INTEGER PRIMARY KEY,
    material_code TEXT,
    year_month TEXT,  -- Format: 2024-01
    consumption_qty REAL,
    consumption_value REAL,
    movement_type TEXT,
    document_reference TEXT,
    FOREIGN KEY (material_code) REFERENCES material_master(material_code)
);

-- Supplier Performance Metrics
CREATE TABLE supplier_performance (
    id INTEGER PRIMARY KEY,
```

```sql
    supplier_code TEXT,
    metric_period TEXT,  -- Monthly: 2024-01
    total_pos INTEGER,
    on_time_deliveries INTEGER,
    late_deliveries INTEGER,
    avg_delay_days REAL,
    quality_rejections INTEGER,
    total_spend REAL,
    performance_score REAL,
    FOREIGN KEY (supplier_code) REFERENCES supplier_master(supplier_code)
);

-- Production Orders (Optional)
CREATE TABLE production_orders (
    order_number TEXT PRIMARY KEY,
    material_code TEXT,
    planned_quantity REAL,
    actual_quantity REAL,
    order_start_date DATE,
    order_end_date DATE,
    status TEXT,
    FOREIGN KEY (material_code) REFERENCES material_master(material_code)
);

-- Alerts & Notifications
CREATE TABLE alerts (
    id INTEGER PRIMARY KEY,
    alert_type TEXT,
    severity TEXT,
    material_code TEXT,
    alert_message TEXT,
    created_date TIMESTAMP,
    is_resolved BOOLEAN,
    resolved_date TIMESTAMP
);
```

---

10. METRICS & KPIs TO TRACK

Inventory Health Metrics

1. **Inventory Turnover Ratio** = COGS / Average Inventory Value
2. **Days of Inventory on Hand (DOH)** = (Current Stock / Avg Daily Consumption)
3. **Stockout Rate** = (Materials with Stock = 0) / Total Materials
4. **Excess Stock Value** = Sum(Stock > Max Level) × Unit Price

5. **Dead Stock Value** = Sum(No movement in 365 days) × Unit Price

Procurement Metrics

1. **On-Time Delivery Rate** = On-time POs / Total POs
2. **Average Lead Time Variance** = Avg(Actual - Promised Lead Time)
3. **Purchase Price Variance (PPV)** = (Standard Price - Actual Price) × Qty
4. **Supplier Defect Rate** = Rejected Qty / Total Received Qty
5. **PO Cycle Time** = Avg(PO Approval Date - PR Creation Date)

Consumption Metrics

1. **Consumption Forecast Accuracy** = 1 - (|Forecast - Actual| / Actual)
2. **ABC Classification Coverage**: % of value in A items
3. **Consumption Volatility (CoV)** = Std Dev / Mean
4. **Safety Stock Adequacy** = Safety Stock / (Lead Time × Avg Consumption)

Agent Performance Metrics

1. **Query Success Rate**: % of queries answered correctly
2. **Average Response Time**: Seconds per query
3. **User Satisfaction Score**: 1-5 rating
4. **Query Complexity Distribution**: Simple/Medium/Complex

---

11. RISK MITIGATION

Technical Risks

Risk	Impact	Probability	Mitigation
Infor API access denied	High	Medium	Get IT approval early; Use ODBC fallback
Data quality issues	High	High	Implement validation layer; Start with data profiling
Query performance slow	Medium	Medium	Use caching; Create indexes; Materialized views
LLM hallucination	High	Medium	Validate SQL before execution; Add human review for critical actions

Risk	Impact	Probability	Mitigation
Cost overruns (API calls)	Low	Low	Use local models for dev; Cache frequent queries

Business Risks

Risk	Impact	Probability	Mitigation
User adoption low	High	Medium	Involve users early; Training sessions; Show quick wins
Accuracy not trusted	High	Low	Show SQL transparency; Start with non-critical queries
Data privacy concerns	High	Low	Role-based access; Audit logging; No PII exposure
Scope creep	Medium	High	Strict POC scope; Phased rollout; Change control

---

12. SUCCESS CRITERIA

POC Success Metrics (Week 4)

- ✅ **Functional**: Answer 20/20 test queries correctly
- ✅ **Performance**: < 5 second response time for 95% queries
- ✅ **Accuracy**: >90% user satisfaction in testing
- ✅ **Technical**: Zero downtime during demo week
- ✅ **Business**: Identify at least 5 actionable insights

Production Readiness Criteria (Week 8-12)

- ✅ **Data Integration**: Live Infor connection with daily refresh
- ✅ **Scalability**: Handle 100 concurrent users
- ✅ **Security**: Role-based access control implemented
- ✅ **Reliability**: 99% uptime SLA
- ✅ **ROI**: Demonstrate 20% time savings in reporting tasks

---

13. NEXT STEPS - YOUR ACTION PLAN

Immediate (This Week)

1. **Get IT Clearance**:
   - Request Infor database access (read-only)
   - Get list of relevant tables and fields
   - Understand data refresh schedules
2. **Stakeholder Alignment**:
   - Meet with procurement head
   - Understand top 10 pain points
   - Prioritize must-have vs nice-to-have features
3. **Set Up Dev Environment**:
   - Install Python 3.10+
   - Install dependencies: `langchain`, `streamlit`, `pandas`, `plotly`, `anthropic`
   - Get Claude API key

Week 1-2: Data & Basic Agent

1. Run the data preparation script (I'll provide)
2. Build SQLite database from Excel
3. Test basic LangChain SQL Agent
4. Create Streamlit UI
5. Demo to 2-3 friendly users

Week 3-4: Enhancements

1. Add forecasting models
2. Build alert system
3. Create dashboard view
4. Prepare production integration specs
5. Final POC demo to stakeholders

---

14. ESTIMATED EFFORT & RESOURCES

Team Required

- **You (Data Scientist/ML Engineer)**: 60% time for 4 weeks

- **Backend Developer**: 20% time (DB setup, API integration)
- **IT Support**: 10% time (Infor access, troubleshooting)
- **Business Analyst**: 10% time (Requirement validation)

Budget (POC only)

- **Claude API**: ~$50-100 (10,000 queries @ $0.01/query)
- **Cloud Hosting** (Optional): $0 (local development) to $100 (AWS/GCP)
- **Tools**: $0 (all open-source for POC)

Total POC Cost: < $200

Time Estimate

- **POC (Weeks 1-4)**: 80-100 hours
- **Production Migration (Weeks 5-12)**: 200-250 hours

---

15. REFERENCES & LEARNING RESOURCES

LangChain & Agents

- LangChain SQL Agent Tutorial
- Building Agents with Claude

Infor Integration

- Infor Documentation Portal (requires login)
- Infor ION API Reference
- Infor IPA User Guide

Time-Series Forecasting

- Facebook Prophet: prophet.plot.ly
- Statsmodels: ARIMA, SARIMA models

Streamlit Dashboards

- Streamlit Gallery
- Streamlit + LangChain Examples

APPENDIX A: SAMPLE CODE SNIPPETS

A1: Load Excel to SQLite

```python
```

```python
import pandas as pd
import sqlite3

# Read Excel files
df1 = pd.read_excel('21c_3yr_MRP_items_consumption.xlsx', header=1)
df2 = pd.read_excel('Lead_Time__21c_3yr_MRP_items_consumption.xlsx', header=1)

# Rename columns
df1.columns = ['sr_no', 'material_code', 'description', 'fy_2022_23',
               'fy_2023_24', 'fy_2024_25', 'total_3yr', 'safety_stock']
df2.columns = ['sr_no', 'material_code', 'description', 'fy_2022_23',
               'fy_2023_24', 'fy_2024_25', 'total_3yr', 'safety_stock', 'lead_time_d

# Merge on material_code
df_master = df1.merge(df2[['material_code', 'lead_time_days']], on='material_code')

# Create SQLite database
conn = sqlite3.connect('procurement.db')

# Save to database
df_master.to_sql('material_master', conn, if_exists='replace', index=False)

# Create consumption history table
consumption_data = []
for _, row in df_master.iterrows():
    consumption_data.append({
        'material_code': row['material_code'],
        'fiscal_year': '2022-23',
        'consumption_qty': row['fy_2022_23']
    })
    consumption_data.append({
        'material_code': row['material_code'],
        'fiscal_year': '2023-24',
        'consumption_qty': row['fy_2023_24']
    })
    consumption_data.append({
        'material_code': row['material_code'],
        'fiscal_year': '2024-25',
        'consumption_qty': row['fy_2024_25']
    })

df_consumption = pd.DataFrame(consumption_data)
df_consumption.to_sql('consumption_history', conn, if_exists='replace', index=False)

conn.close()
```

A2: Basic LangChain SQL Agent

```python
from langchain.agents import create_sql_agent
from langchain.agents.agent_toolkits import SQLDatabaseToolkit
from langchain.sql_database import SQLDatabase
from langchain_anthropic import ChatAnthropic

# Connect to database
db = SQLDatabase.from_uri("sqlite:///procurement.db")

# Initialize Claude
llm = ChatAnthropic(model="claude-sonnet-4-20250514", temperature=0)

# Create SQL toolkit
toolkit = SQLDatabaseToolkit(db=db, llm=llm)

# Create agent
agent = create_sql_agent(
    llm=llm,
    toolkit=toolkit,
    verbose=True,
    agent_type="openai-tools"
)

# Example usage
query = "Show me top 10 materials by consumption in 2024-25"
result = agent.run(query)
print(result)
```

A3: Streamlit UI

```python
```

```python
import streamlit as st
from langchain_anthropic import ChatAnthropic
from langchain.sql_database import SQLDatabase
from langchain.agents import create_sql_agent

st.title("📊 Procurement Analytics Agent")

# Initialize database
@st.cache_resource
def init_agent():
    db = SQLDatabase.from_uri("sqlite:///procurement.db")
    llm = ChatAnthropic(model="claude-sonnet-4-20250514")
    toolkit = SQLDatabaseToolkit(db=db, llm=llm)
    agent = create_sql_agent(llm=llm, toolkit=toolkit, verbose=True)
    return agent

agent = init_agent()

# User input
user_query = st.text_input("Ask a question about procurement data:")

if user_query:
    with st.spinner("Analyzing..."):
        result = agent.run(user_query)
        st.write("### Answer:")
        st.write(result)
```

CONCLUSION

This framework provides you with:

1. ✅ Clear understanding of what you can build NOW vs what needs more data
2. ✅ Step-by-step POC implementation plan
3. ✅ Data gaps identified with Infor extraction specs
4. ✅ Production-ready architecture blueprint
5. ✅ Risk mitigation strategies
6. ✅ Success metrics

Your immediate next action: Review this with your IT team to get Infor database access for the Priority 1 tables listed in Section 4.

Questions to resolve before starting:

1. Do you have Infor database access (ODBC/API)?
2. Who are the business stakeholders for this POC?
3. What's the timeline for production deployment?
4. Any data privacy/security constraints?

I'll help you build the actual POC code once you're ready to proceed!